

Encodings used and their description:

Debug states:

States	Encoding (one hot)	Decimal value
idle_state	7'b00000001	1
move_up_state	7'b00000010	2
move_down_state	7'b00000100	4
door_open_state	7'b00001000	8
door_close_state	7'b00010000	16
emergency_state	7'b00100000	32
rst_state	7'b10000000	64

Floors:

Floor Number	Encoding (binary)	Decimal value
ground	2'b00	0
first	2'b01	1
second	2'b10	2
third	2'b11	3

Hall requests:

Hall request	Encoding (one hot)	Decimal value
Up at ground floor	6'b0000001	1
Up at first floor	6'b0000010	2
Up at second floor	6'b0000100	4
Down at third floor	6'b0001000	8
Down at second floor	6'b0010000	16
Down at first floor	6'b0100000	32

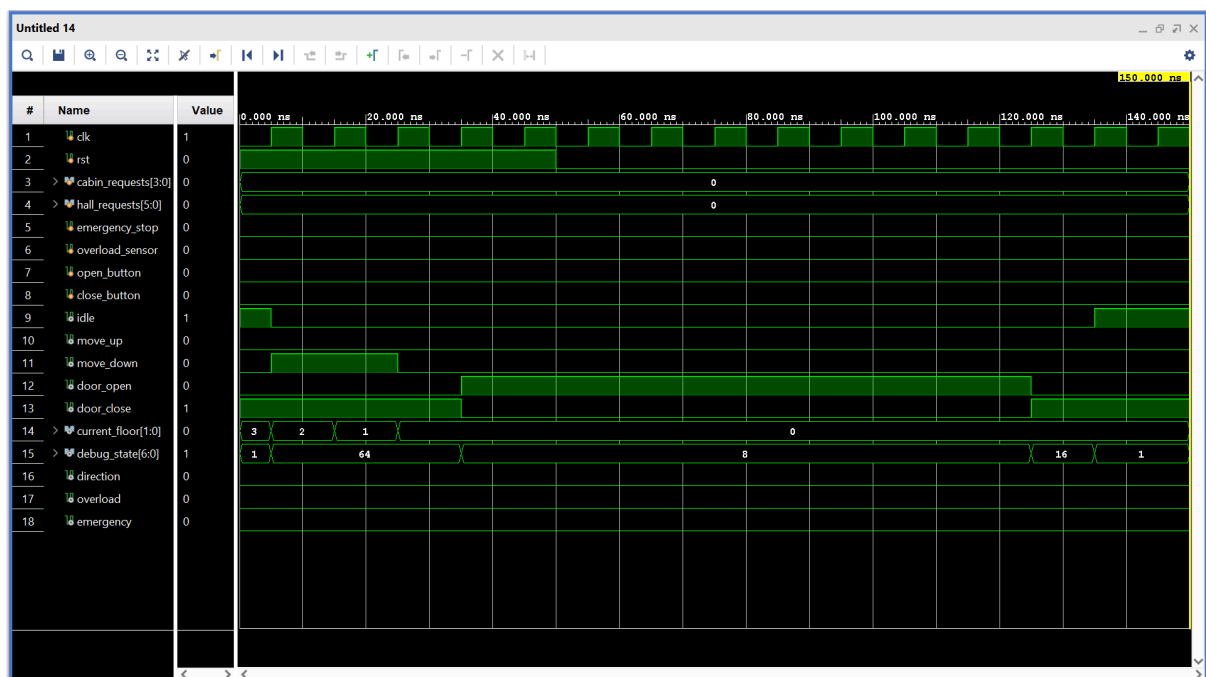
Direction:

Direction	Encoding	Decimal value
Up	1'b1	1
Down	1'b0	0

Case 1: Reset / homing behavior

Start the lift from a non-ground floor, apply reset, and check whether the lift safely reaches ground floor and then goes to idle.

Expected check: The lift should not teleport. It should move down floor by floor until ground and then enter idle state.



Explanation: We started from floor = 2'b11 i.e. 3rd floor. The lift started in idle state, then at the 1st posedge of clk, it encountered rst = 1. Then the lift started to move down to its home floor (ground floor). An important observation is that the lift is not teleporting directly from 3rd floor to gnd floor. It is moving down floor by floor (doors are not opening at 2nd and 1st floor indicating that the lift is just passing by). When it reaches the gnd floor, the doors open for 8 clk cycles and then after that the doors close. Then the lift returns to idle state. While moving down, the move_down signal goes high and an important observation is that the lift moves only when door_close = 1 and hence the doors are never open while moving.

Floor movement sequence: 3 -> 2 (doors not open) -> 1 (not open) -> 0 (doors open)

State transition sequence: idle -> reset -> doors open -> doors close -> idle

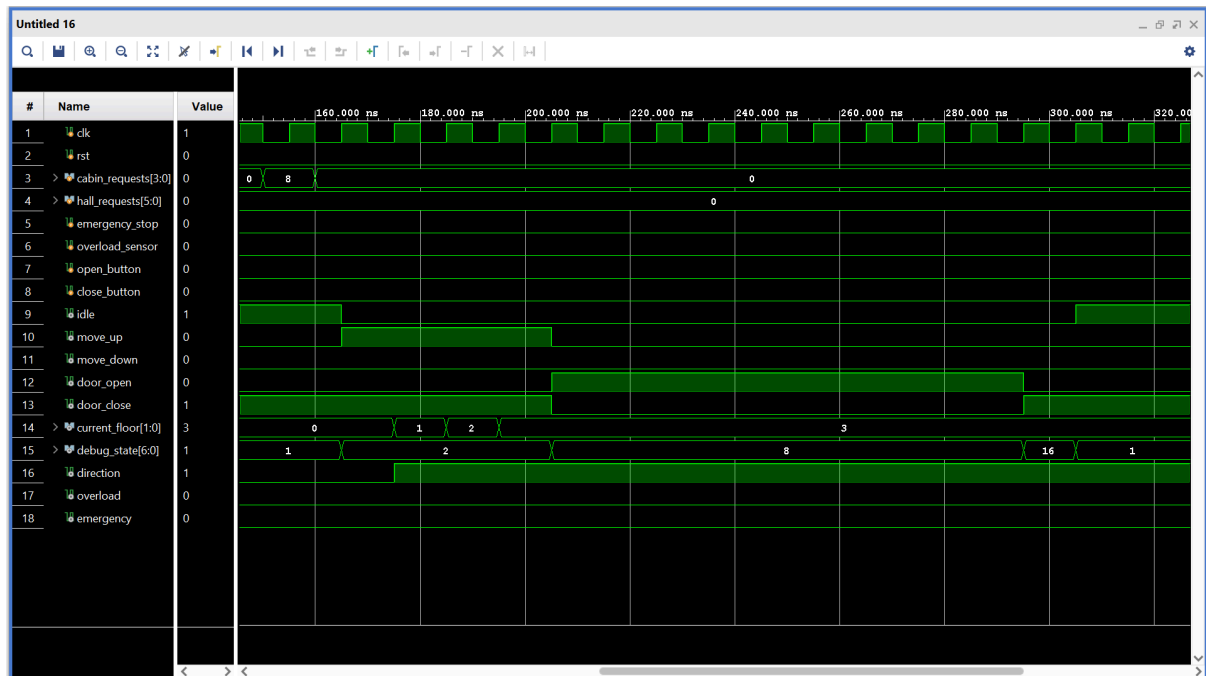
Case Working Status: PASS

Case 2: Single cabin request upward

Start from the ground floor. Give cabin request for the 3rd floor.

Expected check:

The lift should move upward, stop at the 3rd floor, open the door, wait for the door timer/close button, clear the request, and return to idle.



Explanation: The lift was initially at the ground floor due to the previous reset. Then we gave it a cabin request = 4'b1000 i.e. 3rd floor. We had to make cabin request = 0 after a clk cycle else the lift would think that it is getting requested continuously. We observe that the lift started to move up at the posedge just after it encountered the cabin request, while the doors were closed (move_up signal got high only when door_close = 1). Then the lift bypassed through 1st and 2nd floor, while not opening at them. Then when the lift reached the 3rd floor, the doors opened for 8 clk cycles. Further, the doors closed after the completion of the door timer and returned to idle state at last.

Floor movement sequence: 0 -> 1 (doors not open) -> 2 (not open) -> 3 (doors open)

State transition sequence: idle -> move up state -> doors open -> doors close -> idle

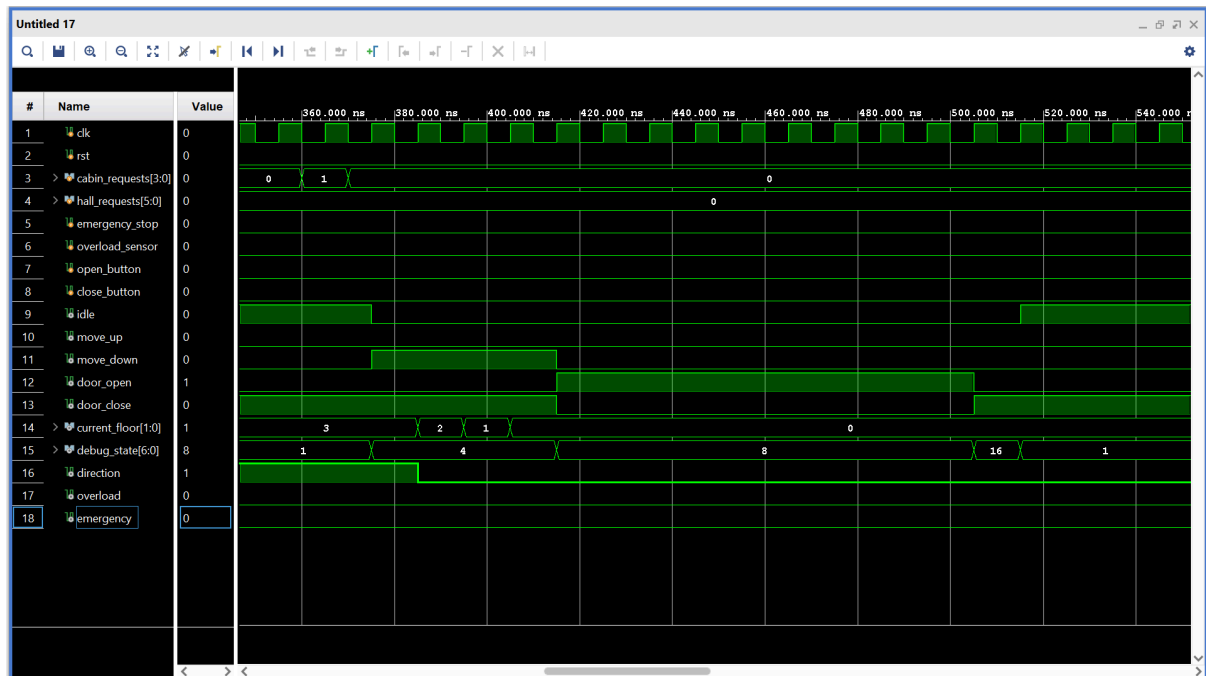
Case Working Status: PASS

Case 3: Multiple cabin requests in same direction

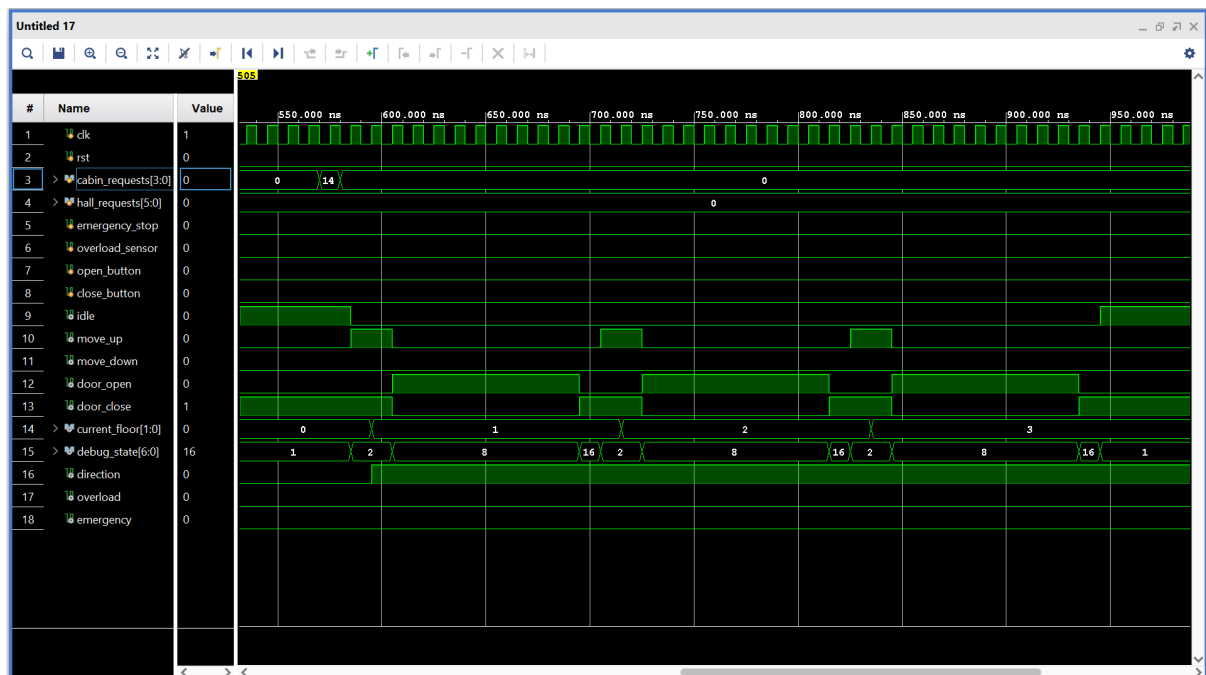
Start from the ground floor. Give cabin requests for 1st, 2nd, and 3rd floors together.

Expected check:

The lift should stop at each requested floor in order: $1 \rightarrow 2 \rightarrow 3$. It should open the door at every requested floor and clear each request after serving.



Since we wanted to start from gnd floor and our lift was the the third floor due to previous case, first we need to get it to the gnd floor. So we ordered a cabin request for gnd floor, `cabin_requests = 4'b0001`. Starting from the 3rd floor, while the doors are closed, the lift starts to move down, bypassing floor 2 and 1. Then when it reaches the gnd floor, doors open for 8 clk cycles and then the doors close going to idle state. Now we are at gnd floor.



Explanation: since we want to request 1st, 2nd and 3rd floors together, we order a cabin req of 1110 (decimal 14). The lift was initially at idle state when we got it to the gnd floor, and the doors were closed, when it gets a cabin request, it starts to move up. When we reach the first floor, the doors open for 8 clk cycles and then close, this time the lift doesn't go to idle state because it has just cleared the request for 1st floor. It still has pending requests for the 2nd and 3rd floor. Then when the doors closed, the lift again started to move up and when it reached the 2nd floor, the doors opened again because that floor was requested. Then this request clears and the doors close. The lift still has a pending request for the 3rd floor. So, it again started to move up and when it reached 3rd floor, the doors opened for 8 clk cycles, and when the timer ended, doors closed and it returned to idle state this time because there were no more pending requests. Another thing to notice is that when we were getting the lift down, the direction was down (0). And when we started to move up from the gnd floor, the direction turned to up (1).

Floor movement sequence: 0 -> 1 (doors open) -> 2 (doors open) -> 3 (doors open)

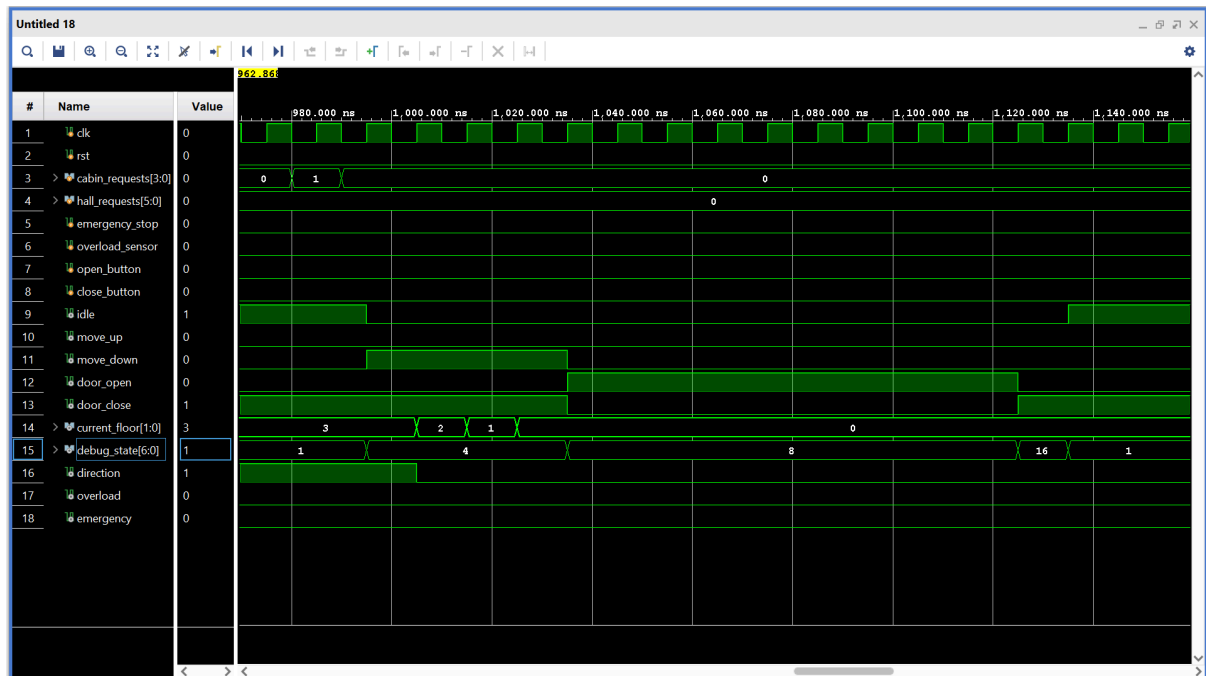
State transition sequence: idle -> move up state -> doors open -> doors close -> move up state -> doors open -> doors close -> move up state -> doors open -> doors close -> idle.

Case Working Status: PASS

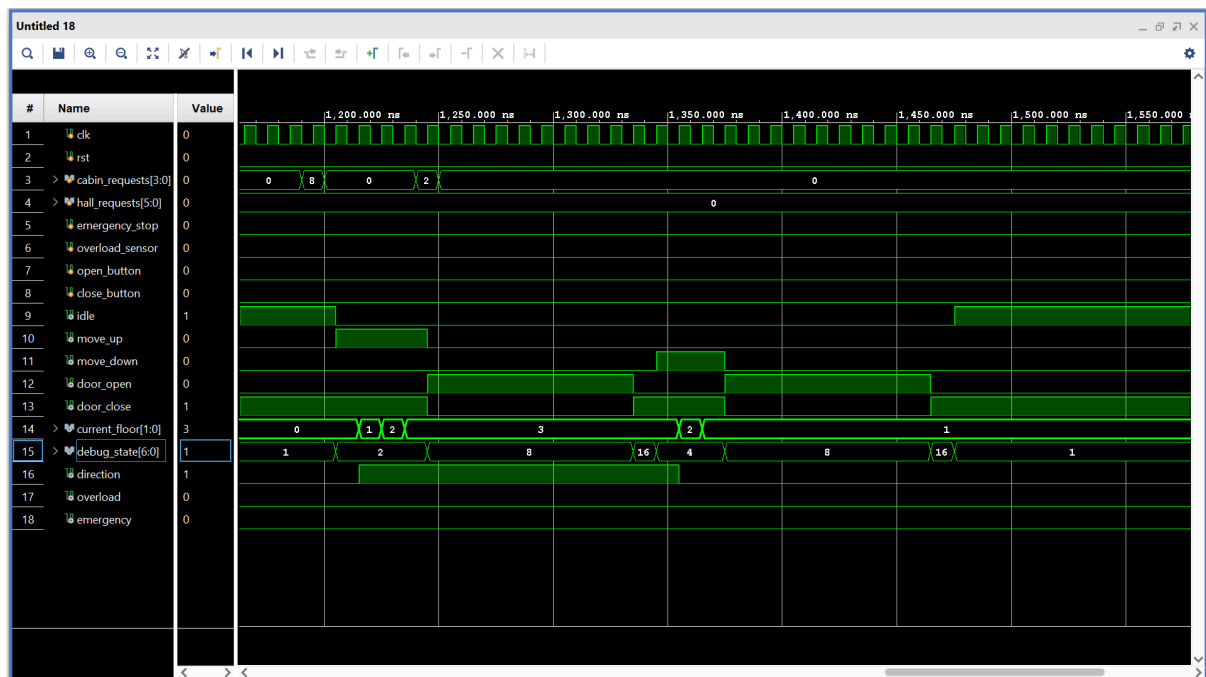
Case 4: Direction priority with mixed requests

Start from ground floor. Give request for 3rd floor. While the lift is moving up, add a request for 1st floor.

Expected check: The lift should continue upward first, serve the upper request, then reverse direction and serve the lower request.



Getting back to the ground floor as done previously.



Explanation: since we returned to the ground floor, we were idle initially. Then, we got an upward cabin_req = 4'b1000 (8), which is the 3rd floor. So, immediately at the next posedge

clk, we started to move up. By the time we had crossed 1st floor, we also got another cabin_req = 4'b0010 (2), which is basically the 1st floor. So the lift continued to move up to serve the 3rd floor request. When it reached the third floor, the doors opened for 8 clk cycles and when the timer expired, the doors closed. Immediately it reversed its direction because it remembered that it also had a pending request down. Then, it started to move down (move_down = 1), bypassed the 2nd floor, and when it reached the 1st floor, doors opened. When the timer expired, the doors closed and this time no more requests were pending. So it went to idle state at last. Another thing to notice is that when the lift was going up to the 3rd floor, its direction changed to up (1). Earlier when we brought the lift to the ground floor from the 3rd floor, its direction was down (0). Now it started to move up for 3rd floor, so its direction was reversed to up. Then finally when it fulfilled 3rd floor, it had to come down and its direction again reversed

Floor movement sequence: 0 -> 1 (doors not open) -> 2 (doors not open) -> 3 (doors open) -> 2 (doors don't open) -> 1 (doors open)

State transition sequence: idle -> move up state -> doors open -> doors close -> move down state -> doors open -> doors close -> idle.

Case Working Status: PASS.

Case 5: Hall request direction handling

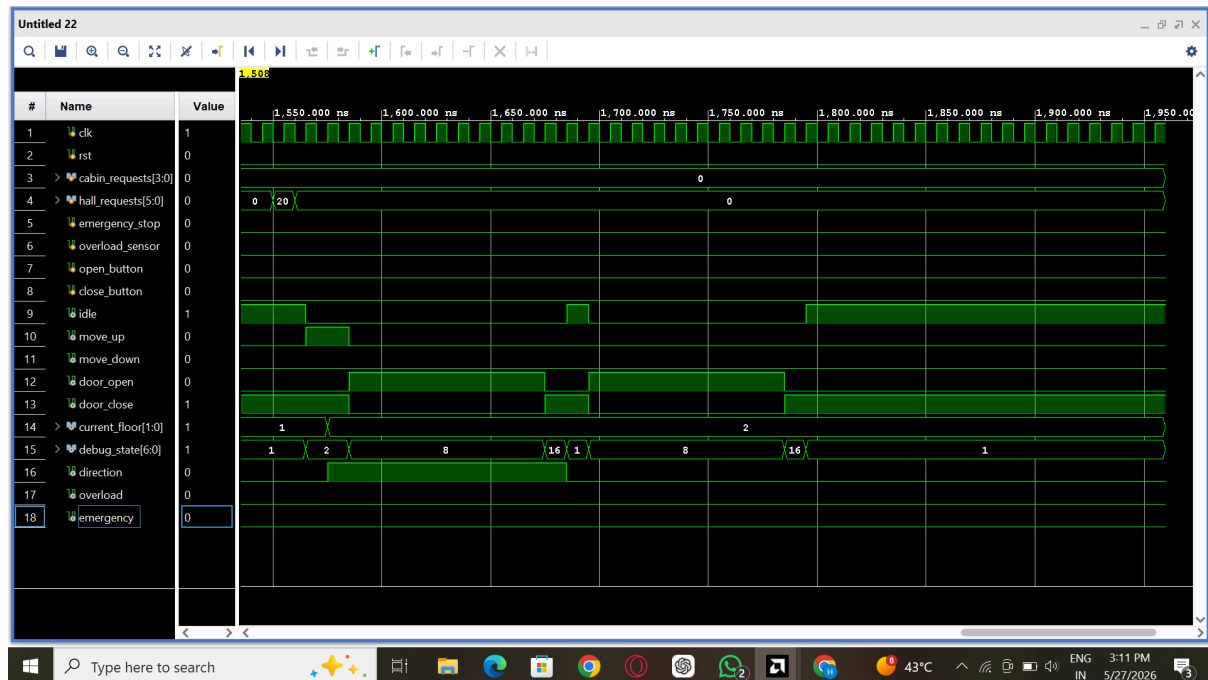
At 2nd floor, create both hall requests: up at 2nd floor and down at 2nd floor. Test once when direction is up and once when direction is down.

Expected check:

When moving up, only the up hall request should be served/cleared.

When moving down, only the down hall request should be served/cleared.

The opposite-direction request should remain pending.



Explanation: due to the last case, we were on the 1st floor now. Then we give hall req = 6'b010100 (20 decimal). This hall req signifies up call at 2nd floor as well as down call at 2nd floor. So since our lift was at 1st floor and it was called above 1st floor, its direction becomes up (1). So the lift starts moving up. It encounters a current request of up at 2nd floor so the lift opens and then closes after 8 clk cycles. Then the request of up was cleared, but one more request remains pending in the opposite direction, 2nd down. So since no more pending requests are there in the current moving direction, the lift changes its direction to move down. Now the lift is at 2nd floor only and it remembers that it gets another down request that was not cleared, so the doors open again while we are at the 2nd floor itself. This will clear the 2 down request also, no more pending requests, the lift goes idle after closing the doors. The important thing to notice is that the lift changes its direction when it is opening the doors to serve the requests.

Floor movement sequence: 1 (doors not open) -> 2 (doors open) -> 2 (doors close) -> 2 (doors open) -> 2 (doors close)

State transition sequence: idle -> move up state -> doors open -> doors close -> idle -> doors open -> doors close -> idle.

Case Working Status: PASS.

Case 6: Door open / close button behavior:

At any floor, press "open_button". Then test:

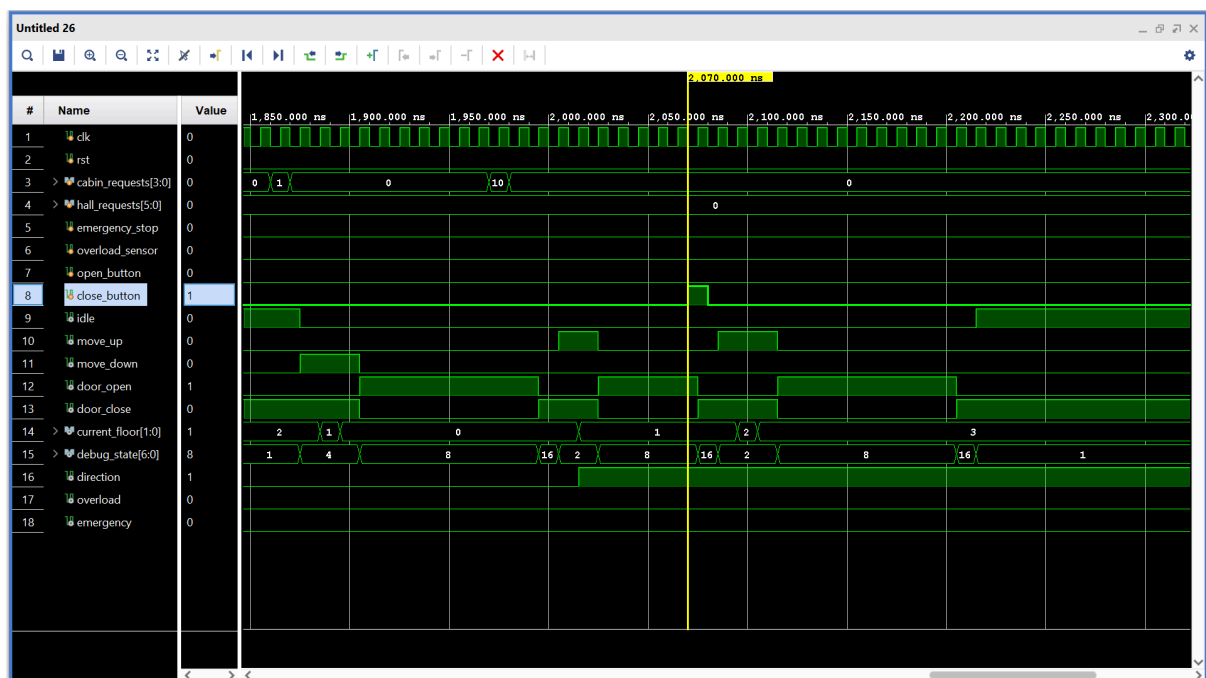
- door remains open until timer expires
- "close_button" closes the door early
- pressing "open_button" during "door_close_state" reopens the door

Expected check:

Door states should behave correctly without losing pending requests.

Method: For door close button testing, we first try to transition to any floor and while the door is open, we press the door close button, and the doors should close before the default 8 clk cycle timer.

For door open button testing, we wait for the doors to close at a floor, then press the open button and the lift doors should open.

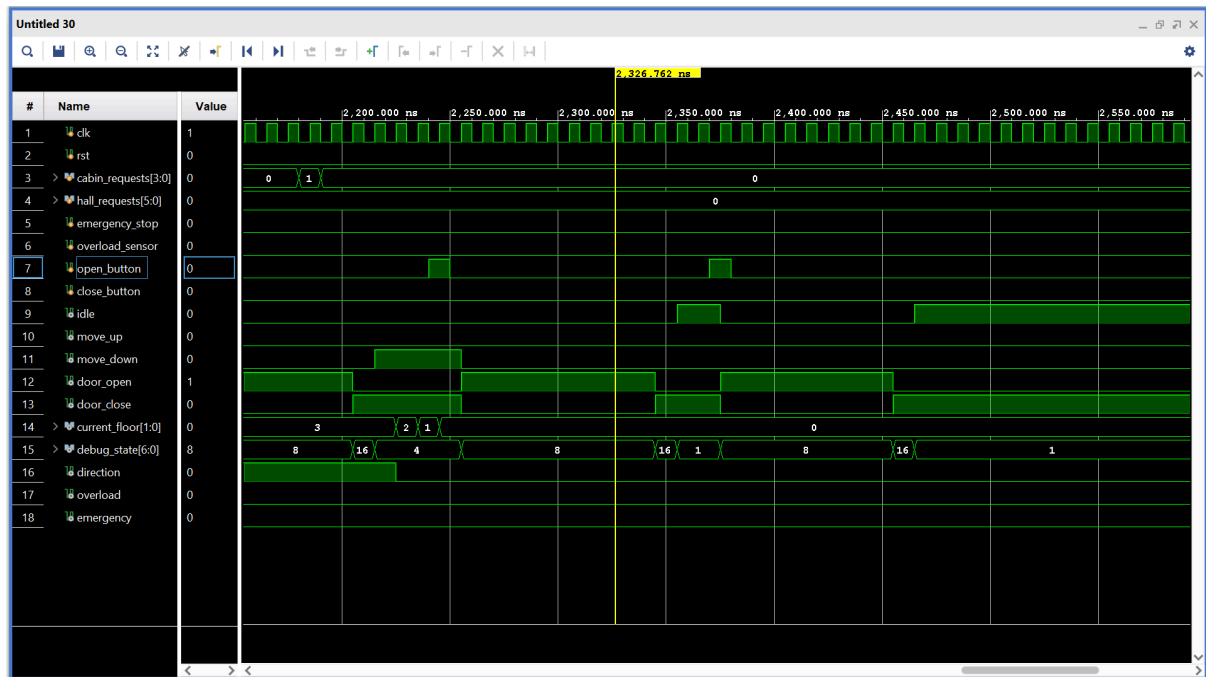


Explanation for close button: We went to the gnd floor, then gave the cabin request = 1010 (10). So while the lift was moving up to fulfil the requests and it opened the doors at 1st floor, we pressed the close button. We can see that the doors closed before the timer ended (before 8 clk cycles only). Another important thing to note is that even if the doors closed earlier, the lift went up to fulfil the request at 3rd floor and hence, it did not lose the pending requests.

Floor movement sequence: 2 (doors not open) -> 1 (doors not open) -> 0 (doors open) -> 1 (doors open, then forced to close) -> 2 (doors not open) -> 3 (doors open)

State transition sequence: idle -> move down state -> doors open -> doors close -> move up -> doors open -> doors close -> move up -> doors open -> doors close -> idle.

Case Working Status: PASS.



Explanation for open button: after reaching the 3rd floor, we gave a cabin request for gnd floor. We gave an open button command twice, once while the lift was moving and once when the lift reached gnd floor. We can observe that while the lift was moving and the open button command was asserted, the doors did not open, but it continued its operation which indicates it did not forget the pending request. On the other hand when we reached gnd floor and we gave open button command, the lift doors opened for 8 clk cycles, then the doors closed and the lift went idle. This ensures a safety interlock so that the lift doesn't open while it is moving.

Floor movement sequence: 3 (doors not open) -> 2 (doors not open) -> 1 (doors close) -> 0 (doors open) -> 0 (doors close) -> 0 (doors forced to open) -> 0 (doors close)

State transition sequence: move down state -> doors open -> doors close -> idle -> doors open -> doors close -> idle.

Case Working Status: PASS.

Case 7: Safety case: emergency and overload

Test two separate safety scenarios:

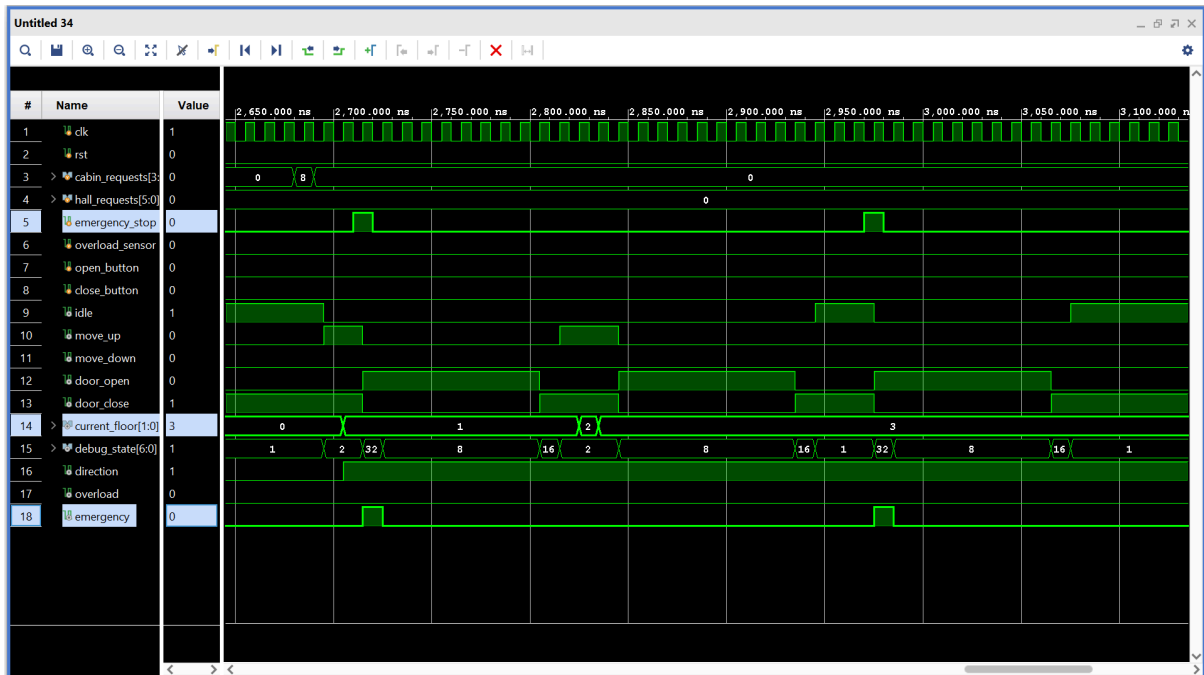
A. Assert "emergency_stop" while the lift is moving.

Expected: movement should stop, emergency output should assert, and door should open.

B. Assert "overload_sensor" while the door is open or closing.

Expected: door should remain/open, movement should not start, and overload output should assert.

Emergency stop:



Explanation: we were lastly at the gnd floor, we gave a cabin request for the 3rd floor.

Ideally what should have happened is that the list crosses floor 1, floor 2 and go and open at 3rd floor. Then, after completing the timer, the doors close and the lift becomes idle.

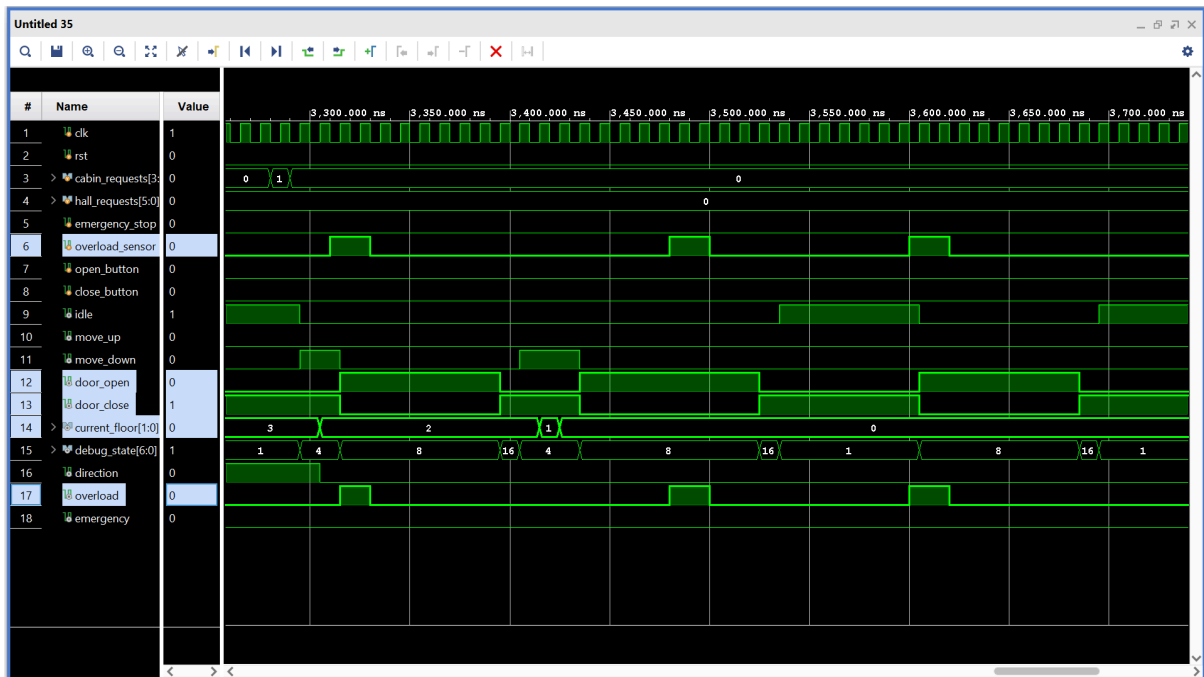
But now we asserted emergency stop while the lift was moving, so the lift stopped at the nearest floor (1st floor in this case) and the doors opened at that floor. The timer completed after the emergency stop was deasserted and then normal motion for floor 3 continued. Now when the lift reached the third floor, opened its doors, closed it after the timer and went idle, again an emergency stop occurred. Then, the lift again opened at that floor and remained open till the timer expired after deassertion of the emergency stop. Also, the emergency output goes high when there is an emergency stop.

Floor movement sequence: 0 -> 1 (doors forced to open due to emergency) -> 2 (doors close) -> 3 (doors open) -> 3 (doors close) -> 3 (doors forced to open due to emergency) -> 3 (doors close).

State transition sequence: idle -> move up state -> emergency -> doors open -> doors close -> move up -> doors open -> doors close -> idle -> emergency -> doors open -> doors close -> idle.

Case Working Status: PASS.

Overload:



Explanation: we were lastly at 3rd floor, then we gave a cabin request for gnd floor. Ideally, the lift should go from floor 3 -> 2 -> 1 -> 0, while the lift not opening at floor 1 and 2. But while the lift was moving down, it detected an overload. So, it immediately stopped at the nearest floor, floor 2. The doors opened there till the timer didn't expire after deassertion of the overload sensor. Then the doors closed and it tried to complete its pending request to go to gnd floor. Once again when the doors were open and it detected overload, the doors remained open. After the request was cleared at the gnd floor, the lift doors closed and it went idle. Then once again it detected overload. The doors opened again and closed after the timer expired on deassertion of the overload sensor. While the overload sensor was high, the overload output was also asserted high.

Floor movement sequence: 3 -> 2 (doors forced to open due to overload) -> 1 (doors close) -> 0 (doors open) -> 0 (doors close) -> 0 (doors forced to open due to overload) -> 0 (doors close).

State transition sequence: idle -> move down state -> doors open -> doors close -> move down -> doors open -> doors close -> idle -> doors open -> doors close -> idle.

Case Working Status: PASS.